

TAKE-HOME ASSIGNMENT

Research Summarizer Agent

Multi-Agent System with OpenAI MCP Integration

Target Level: Software engineer	Duration: 1 Week from Date of Issue
Domain: Generative AI / Agentic Systems	Language: Java (preferred) or Python

1. Background

Modern AI-powered applications often need to gather information from multiple sources, extract meaningful insights, and present them as a coherent report — all autonomously. This assignment asks you to build exactly that: a pipeline of three collaborating agents that takes a research topic as input and delivers a structured summary report as output.

A key requirement of this system is intelligent source routing. OpenAI maintains a public MCP (Model Context Protocol) server at:

```
https://developers.openai.com/mcp
```

This server provides read-only, searchable access to OpenAI's developer documentation — covering the API, Agents SDK, Models, Responses API, and more. Your Search Agent must prefer this MCP server when the query is OpenAI developer-related, and fall back to a general web search only when the MCP server does not return sufficient results.

2. Objective

Design and implement a multi-agent Research Summarizer system. The system must:

- ▶ Accept a research topic via a REST API endpoint.
- ▶ Route the query through a pipeline of three specialised agents.
- ▶ Produce a well-structured summary report as the final output.
- ▶ Intelligently choose between the OpenAI Docs MCP server and general web search based on query intent.

3. Agent Architecture

You must implement exactly three agents. Each agent has a single responsibility and hands off its output to the next.

Agent 1 — Search Agent

Role: Receive the user query. Determine if it is OpenAI developer-related. If yes, query the OpenAI Docs MCP server first. If the MCP response is insufficient or empty, fall back to a general web search tool. Return raw search results to Agent 2.

Tools: OpenAI Docs MCP (<https://developers.openai.com/mcp>) | Web Search Tool (e.g. Tavily, SerpAPI, DuckDuckGo, or built-in LLM web search)

Handoff to: Agent 2 — Insight Extractor

Agent 2 — Insight Extractor

Role: Receive raw search results from Agent 1. Extract key facts, statistics, definitions, and notable quotes. Return a structured list of insights to Agent 3.

Tools: LLM extraction (prompt-based)

Handoff to: Agent 3 — Report Generator

Agent 3 — Report Generator

Role: Receive the structured insights from Agent 2. Generate a final, formatted summary report with clear sections: Executive Summary, Key Findings, Details, and Sources.

Tools: LLM text generation | Template rendering (Markdown or JSON output)

Handoff to: REST API Response (final output)

4. OpenAI MCP Routing Logic (Critical Requirement)

The Search Agent must implement the following decision logic:

Query Type	Primary Source	Fallback
OpenAI API / Models / SDK / Agents / Fine-tuning / Responses API	OpenAI Docs MCP Server	Web Search
General / Non-OpenAI topics	Web Search	N/A

The routing decision must be deterministic and documented. The candidate must explain in the README how the agent classifies a query as OpenAI developer-related.

5. REST API Specification

Expose a single REST endpoint to trigger the full pipeline:

POST	<code>/api/research/summarize</code>
Request Body	<code>{ "topic": "string", "maxSources": 5 }</code>
Response	<code>{ "topic", "searchSource" (MCP WEB), "executiveSummary", "keyFindings": [], "details", "sources": [] }</code>
Error Handling	Return appropriate HTTP status codes (400, 500) with a structured error body

6. Technical Requirements

6.1 Mandatory

- ▶ Java based agentic framework or Python based agentic framework
- ▶ MCP client that connects to `https://developers.openai.com/mcp` using the Streamable HTTP transport
- ▶ Three clearly named agent classes/components, each in its own file or package
- ▶ At least one web search tool integration (Tavily, SerpAPI, DuckDuckGo API, or LLM-native web search)
- ▶ README.md covering: setup, architecture diagram (ASCII or image), API contract, and routing logic explanation

6.2 Nice to Have (Bonus Points)

- ▶ Unit tests for the routing logic and at least one agent
- ▶ Docker Compose setup for local run
- ▶ Structured logging with request trace IDs across agent hops

6.3 Constraints

- ▶ The system must work end-to-end with a single curl command after setup
- ▶ No UI is required — this is a backend/API assignment
- ▶ Can use any orchestration platform (LangChain, LangGraph, Google ADK, CrewAI) — to build agent coordination.

7. Deliverables

Submit as a Git repository (GitHub / Bitbucket) containing:

1. **Source Code** — Complete, runnable project with all three agents
2. **README.md** — Setup instructions, architecture overview, routing logic explanation

3. **Sample API Responses** — At least two: one OpenAI developer topic (routed via MCP) and one general topic (routed via Web Search)
4. **Design Decisions** — A short section (can be inside README) explaining key design choices and trade-offs

8. Notes & Guidelines

- ▶ You are free to choose any LLM provider (OpenAI, Anthropic, Gemini, or an open model) for the agent reasoning — document your choice.
- ▶ The OpenAI Docs MCP server is read-only and requires no authentication. Transport: Streamable HTTP.
- ▶ There is no single correct architecture. We are evaluating your reasoning and design choices as much as the code.
- ▶ Partial submissions are acceptable — clearly document what is and is not completed.

Good luck! We look forward to seeing your approach.

Questions? Reach out to the hiring team before you start.